

LAB PROGRAM

CHAPTER 2

1. Write a program to perform the following

- o An empty list
- o A list with one element
- o A list with all identical elements
- o A list with negative

numbers Test Cases:

1. Input: []

o Expected Output: []

2. Input: [1]

o Expected Output: [1]

3. Input: [7, 7, 7, 7]

o Expected Output: [7, 7, 7, 7]

4. Input: [-5, -1, -3, -2, -4]

o Expected Output: [-5, -4, -3, -2, -1]

CODE:

```
def sort_list(lst):
```

```
    return
```

```
    sorted(lst)
```

```
lists = [
```

```
    [],
```

```
    [1],
```

```
    [7, 7, 7, 7],
```

```
    [-5, -1, -3, -2, -4]
```

```
]
```

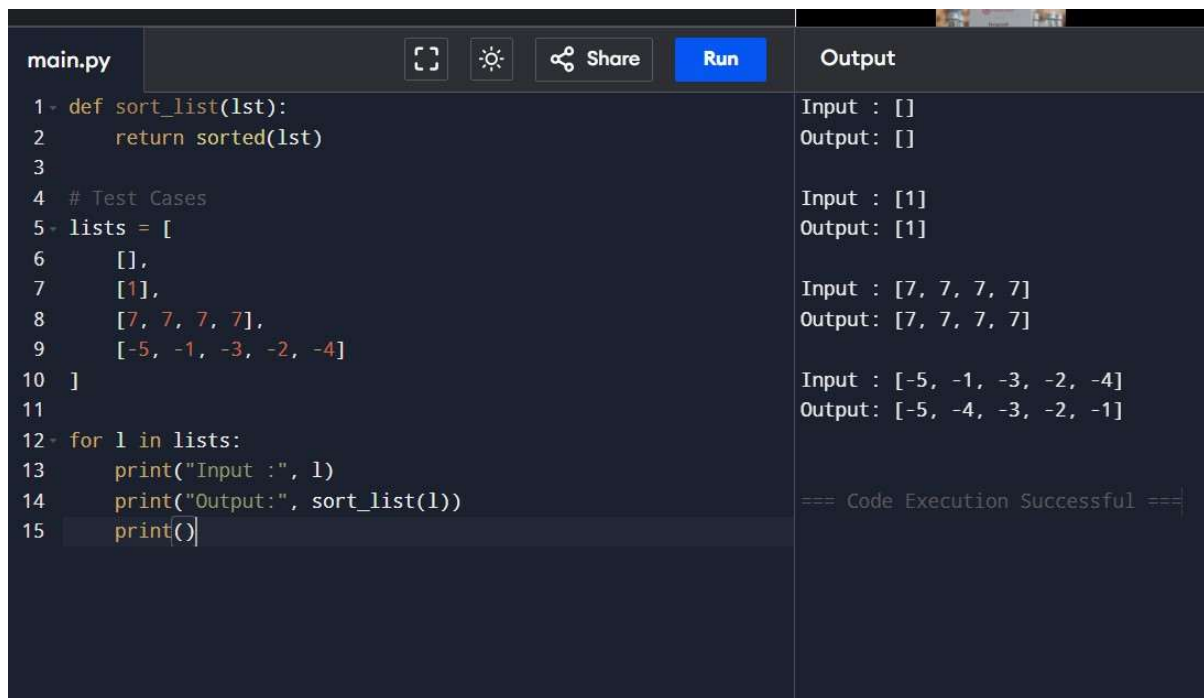
```
for l in lists:
```

```
    print("Input :", l)
```

```
    print("Output:",
```

```
        sort_list(l)) print()
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'sort_list' that returns a sorted list. It then tests this function with four different input lists. The output on the right shows the corresponding sorted lists for each input. The code is as follows:

```
1- def sort_list(lst):
2-     return sorted(lst)
3-
4- # Test Cases
5- lists = [
6-     [],
7-     [1],
8-     [7, 7, 7, 7],
9-     [-5, -1, -3, -2, -4]
10- ]
11-
12- for l in lists:
13-     print("Input :", l)
14-     print("Output:", sort_list(l))
15-     print()
```

The output on the right is:

```
Input : []
Output: []

Input : [1]
Output: [1]

Input : [7, 7, 7, 7]
Output: [7, 7, 7, 7]

Input : [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

=== Code Execution Successful ===
```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted

Array: Input: [1, 2, 3, 4, 5]

CODE

```
def
    selection_sort(arr):
        n = len(arr)
        for i in
            range(n):
                min_index =
                i
                for j in range(i + 1, n):
                    if arr[j] < arr[min_index]:
                        min_index = j
                arr[i], arr[min_index] = arr[min_index],
                arr[i]
        return arr
print(selection_sort([5, 2, 9, 1, 5, 6]))
print(selection_sort([10, 8, 6, 4, 2]))
print(selection_sort([1, 2, 3, 4, 5]))
```

OUTPUT

main.py	Run	Output
<pre>1- def selection_sort(arr): 2- n = len(arr) 3- 4- for i in range(n): 5- min_index = i 6- 7- for j in range(i + 1, n): 8- if arr[j] < arr[min_index]: 9- min_index = j 10- 11- arr[i], arr[min_index] = arr[min_index], arr[i] 12- 13- return arr 14- 15- 16- print(selection_sort([5, 2, 9, 1, 5, 6])) 17- print(selection_sort([10, 8, 6, 4, 2])) 18- print(selection_sort([1, 2, 3, 4, 5]))</pre>		<pre>[1, 2, 5, 5, 6, 9] [2, 4, 6, 8, 10] [1, 2, 3, 4, 5] === Code Execution Successful ===</pre>

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

CODE

```
def
    bubble_sort(arr):
        n = len(arr)

        for i in range(n):
            swapped = False
            for j in range(0, n - i -
                1): if arr[j] > arr[j +
                    1]:
                        arr[j], arr[j + 1] = arr[j + 1], arr[j]
                        swapped = True
            if not swapped:
                break
        return arr
print(bubble_sort([64,25,12,22,11]
))
print(bubble_sort([29,10,14,37,13]
)) print(bubble_sort([3,5,2,1,4]))
print(bubble_sort([1,2,3,4,5]))
print(bubble_sort([5,4,3,2,1]))
```

OUTPUT

main.py	Run	Output
<pre>1- def bubble_sort(arr): 2 n = len(arr) 3 4- for i in range(n): 5 swapped = False 6 7- for j in range(0, n - i - 1): 8 9- if arr[j] > arr[j + 1]: 10 arr[j], arr[j + 1] = arr[j + 1], arr[j] 11 swapped = True 12 13- if not swapped: 14 break 15 16 return arr 17 18 19 print(bubble_sort([64,25,12,22,11])) 20 print(bubble_sort([29,10,14,37,13])) 21 print(bubble_sort([3,5,2,1,4])) 22 print(bubble_sort([1,2,3,4,5])) 23 print(bubble_sort([5,4,3,2,1]))</pre>		<pre>[11, 12, 22, 25, 64] [10, 13, 14, 29, 37] [1, 2, 3, 4, 5] [1, 2, 3, 4, 5] [1, 2, 3, 4, 5] === Code Execution Successful ===</pre>

4. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome. CODE

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key  
    return arr  
  
print(insertion_sort([3,1,4,1,5,9,2,6,5,3]))  
) print(insertion_sort([5,5,5,5,5]))  
print(insertion_sort([2,3,1,3,2,1,1,3]))
```

OUTPUT

main.py	Run	Output
<pre>1- def insertion_sort(arr): 2 3- for i in range(1, len(arr)): 4 5 key = arr[i] 6 j = i - 1 7 8- while j >= 0 and arr[j] > key: 9 arr[j + 1] = arr[j] 10 j -= 1 11 12 arr[j + 1] = key 13 14 return arr 15 16 17 print(insertion_sort([3,1,4,1,5,9,2,6,5,3])) 18 print(insertion_sort([5,5,5,5,5])) 19 print(insertion_sort([2,3,1,3,2,1,1,3]))</pre>		<pre>[1, 1, 2, 3, 3, 4, 5, 5, 6, 9] [5, 5, 5, 5, 5] [1, 1, 1, 2, 2, 3, 3, 3] === Code Execution Successful ===</pre>

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.

Example 2:

Input: arr = [1,2,3,4], k =

2 Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

CODE

```
def findKthPositive(arr, k):
    current = 1
    i = 0
    while k > 0:
        if i < len(arr) and arr[i] ==
            current: i += 1
        else:
            k -= 1
        if k == 0:
            return
        current
        current += 1
print(findKthPositive([2,3,4,7,11],
5))
print(findKthPositive([1,2,3,4],2))
```

main.py	Output
<pre> 1- def findKthPositive(arr, k): 2- current = 1 3- i = 0 4- 5- while k > 0: 6- if i < len(arr) and arr[i] == current: 7- i += 1 8- else: 9- k -= 1 10- if k == 0: 11- return current 12- current += 1 13- 14- 15- print(findKthPositive([2,3,4,7,11],5)) 16- print(findKthPositive([1,2,3,4],2)) </pre>	<pre> 9 6 === Code Execution Successful === </pre>

OUTPUT

6.A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer

array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that $\text{nums}[-1] = \text{nums}[n] = -\infty$. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

CODE

```

def
    findPeakElement(nums
    ): left = 0
    right = len(nums)-1
    while left < right:
        mid = (left+right)//2
        if nums[mid] >
            nums[mid+1]: right = mid
        else:
            left = mid+1
    return left

print(findPeakElement([1,2,3,1]))
print(findPeakElement([1,2,1,3,5,6,4]))

```


OUTPUT

main.py	Output
<pre>1 def findPeakElement(nums): 2 left = 0 3 right = len(nums)-1 4 5 while left < right: 6 mid = (left+right)//2 7 8 if nums[mid] > nums[mid+1]: 9 right = mid 10 else: 11 left = mid+1 12 13 return left 14 15 16 print(findPeakElement([1,2,3,1])) 17 print(findPeakElement([1,2,1,3,5,6,4]))</pre>	<pre>2 5 === Code Execution S</pre>

7. Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

CODE

```
def strStr(haystack, needle):
    return
    haystack.find(needle)
print(strStr("sadbutsad", "sad"))
print(strStr("leetcode", "leeto"))'
```

OUTPUT

main.py	Output
<pre>1 def strStr(haystack, needle): 2 return haystack.find(needle) 3 4 print(strStr("sadbutsad", "sad")) 5 print(strStr("leetcode", "leeto"))</pre>	<pre>0 -1 === Code Execution</pre>

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

CODE

```
def
    stringMatching(words
): ans=[]
    for i in range(len(words)):
        for j in
            range(len(words)):
                if i!=j and words[i] in words[j]:
                    ans.append(words[i])
                    brea
    k return ans

print(stringMatching(["mass","as","hero","superhero"]))
print(stringMatching(["leetcode","et","code"]))
print(stringMatching(["blue","green","bu"]))
```

OUTPUT

main.py	Run	Output
<pre>1- def stringMatching(words): 2- ans=[] 3- 4- for i in range(len(words)): 5- for j in range(len(words)): 6- if i!=j and words[i] in words[j]: 7- ans.append(words[i]) 8- break 9- 10- return ans 11 12 13 print(stringMatching(["mass","as","hero","superhero"])) 14 print(stringMatching(["leetcode","et","code"])) 15 print(stringMatching(["blue","green","bu"]))</pre>		<pre>['as', 'hero'] ['et', 'code'] [] === Code Executing</pre>

9. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

CODE

```
import math

def closest_pair(points):
    min_distance=float('inf')
    pair=None
    for i in range(len(points)):
        for j in range(i+1,len(points)):
            d=math.dist(points[i],points[j])
            if d<min_distance:
                min_distance=d
                pair=(points[i],points[j])
    return pair,min_distance
points=[(1,2),(4,5),(7,8),(3,
1)]
pair,distance=closest_pair(points)
print("Closest Pair:",pair)
print("Minimum
Distance:",distance) OUTPUT
```

main.py	Output
<pre>1 import math 2 3 def closest_pair(points): 4 5 min_distance=float('inf') 6 pair=None 7 8 for i in range(len(points)): 9 for j in range(i+1,len(points)): 10 11 d=math.dist(points[i],points[j]) 12 13 if d<min_distance: 14 min_distance=d 15 pair=(points[i],points[j]) 16 17 return pair,min_distance 18 19 20 points=[(1,2),(4,5),(7,8),(3,1)] 21 22 pair,distance=closest_pair(points) 23 24 print("Closest Pair:",pair) 25 print("Minimum Distance:",distance)</pre>	<pre>Closest Pair: ((1, 2), (3, 1)) Minimum Distance: 2.23606797749979 === Code Execution Successful ===</pre>

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

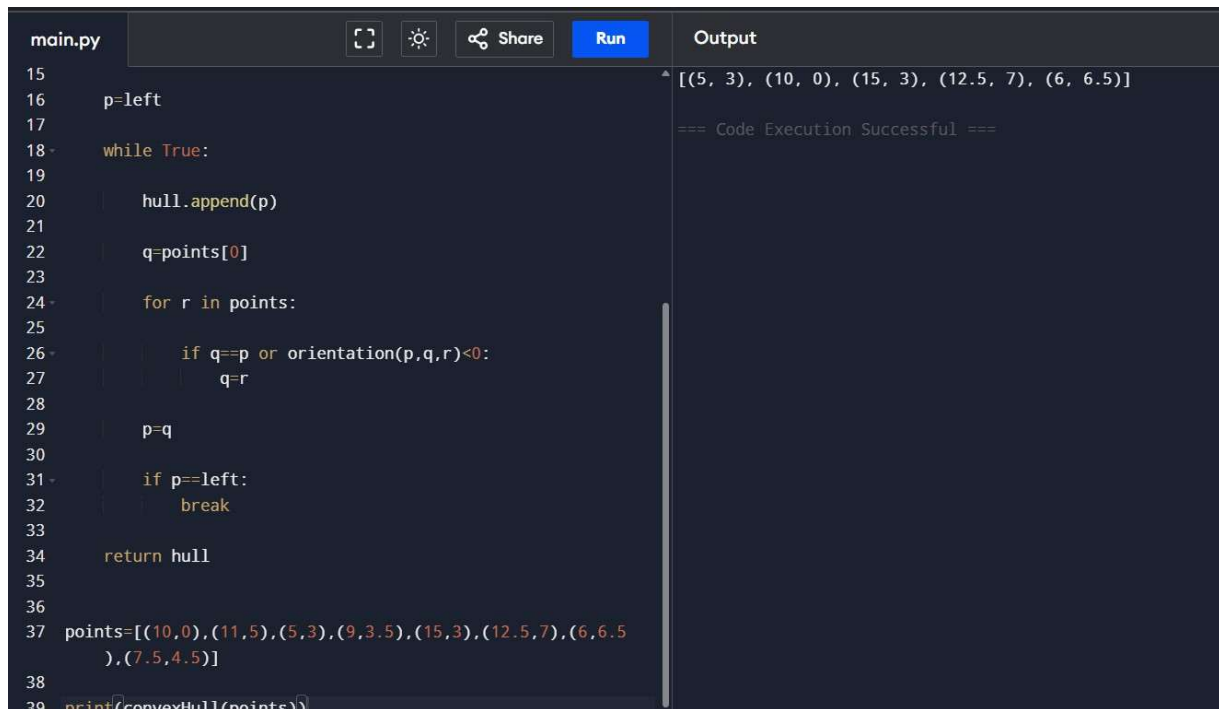
CODE

```
def orientation(p,q,r):
    return (q[0]-p[0])*(r[1]-p[1])-(q[1]-p[1])*(r[0]-
p[0])
def convexHull(points):
    n=len(points)
    if n<3:
        return points
    hull=[]
    left=min(points
)
    p=left
    while True:
        hull.append(p)
        q=points[0]

        for r in points:
            if q==p or orientation(p,q,r)<0:
                q=r
        p=q
    if
        p==lef
    t:
        break
```

```
    return hull
points=[(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5),(7.5,4.5)]
print(convexHull(points))
```

OUTPUT



```
main.py  [Icons]  Run  Output
15
16 p=left
17
18 while True:
19
20     hull.append(p)
21
22     q=points[0]
23
24     for r in points:
25
26         if q==p or orientation(p,q,r)<0:
27             q=r
28
29     p=q
30
31     if p==left:
32         break
33
34     return hull
35
36
37 points=[(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5),
38         (7.5,4.5)]
39 print(convexHull(points))
```

Output

```
[(5, 3), (10, 0), (15, 3), (12.5, 7), (6, 6.5)]
=== Code Execution Successful ===
```

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

CODE

```
def orientation(p,q,r):
    return (q[0]-p[0])*(r[1]-p[1])-(q[1]-p[1])*(r[0]-
p[0])
def convexHull(points):
    left=min(points
)
    hull=[]
    p=left
    while
    True:
        hull.append(
        p)
        q=points[0]
        for r in
        points:
            if q==p or orientation(p,q,r)<0:
                q=r
        p=q
        if
```

p==lef

t:

break

```

    return hull

points=[(1,1),(4,6),(8,1),(0,0),(3,3)
]

print(convexHull(points))

```

OUTPUT

```

main.py  [Icons]  Share  Run  Output
9      hull=[]
10
11      p=left
12
13      while True:
14
15          hull.append(p)
16
17          q=points[0]
18
19          for r in points:
20
21              if q==p or orientation(p,q,r)<0:
22                  q=r
23
24          p=q
25
26          if p==left:
27              break
28
29      return hull
30
31
32 points=[(1,1),(4,6),(8,1),(0,0),(3,3)]
33
34 print(convexHull(points))

```

Output: [(0, 0), (8, 1), (4, 6)]

=== Code Execution Successful ===

12. 1 You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function distance(city1, city2) to calculate the distance between two cities (e.g., Euclidean distance).

2. Implement a function tsp(cities) that takes a list of cities as input and performs the following:

- o Generate all possible permutations of the cities (excluding the starting city) using `itertools.permutations`.

- o For each permutation (representing a potential route):

- Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.

- Keep track of the shortest distance encountered and the corresponding path.

o Return the minimum distance and the shortest path (including the starting city at the beginning and end).

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

CODE

```
from itertools import permutations
from math import dist

def tsp(cities):
    start=cities[0]
    shortest=float('inf')
    best_path=[]
    for perm in permutations(cities[1:]):
        path=[start]+list(perm)+[start]
        distance=0
        for i in range(len(path)-1):
            distance+=dist(path[i],path[i+1])
        if
            distance<shortest:
                shortest=distance
                best_path=path
    return shortest,best_path

cities=[(1,2),(4,5),(7,1),(3,6
)]

d,p=tsp(cities)
print("Shortest
Distance:",d)
```

```
print("Shortest Path:",p)
```

OUTPUT

```
main.py  Run  Share  Output
7
8 shortest=float('inf')
9 best_path=[]
10
11 for perm in permutations(cities[1:]):
12     path=[start]+list(perm)+[start]
13
14     distance=0
15
16     for i in range(len(path)-1):
17         distance+=dist(path[i],path[i+1])
18
19     if distance<shortest:
20         shortest=distance
21         best_path=perm
22
23 return shortest,best_path
24
25
26
27 cities=[(1,2),(4,5),(7,1),(3,6)]
28
29 d,p=tsp(cities)
30
31 print("Shortest Distance:",d)
32 print("Shortest Path:",p)
```

```
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

=== Code Execution Successful ===
```

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions). CODE

```
from itertools import
```

```
permutations def
```

```
assignment_problem(cost):
```

```
    n=len(cost)
```

```
    minimum=float('in
```

```
    f') best=None
```

```
    for perm in permutations(range(n)):
```

```
        total=0
```

```
        for worker in range(n):
```

```
            total+=cost[worker][perm[worker]]
```

```

        if
            total<minimu
            m:
            minimum=tot
            al best=perm
        return
    best,minimum
cost=[[3,10,7],
      [8,5,12],
      [4,6,9]
assignment,total=assignment_problem(c
ost) print("Assignment:",assignment)
print("Total Cost:",total)

```

OUTPUT

The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'assignment_problem' that uses permutations to find the minimum total cost for assigning workers to tasks. The cost matrix is defined as:

```

cost = [[3, 10, 7],
        [8, 5, 12],
        [4, 6, 9]]

```

The function is called with 'cost' as an argument. The output of the program is displayed in the 'Output' panel on the right:

```

Assignment: (2, 1, 0)
Total Cost: 16
=== Code Execution Successful ===

```

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

2. Define a function `is_feasible(items, weights, capacity)` that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity. **CODE**

```
main.py  [ ] [ ] [ ] Share Run Output
1  from itertools import permutations
2
3  def assignment_problem(cost):
4
5      n=len(cost)
6
7      minimum=float('inf')
8      best=None
9
10     for perm in permutations(range(n)):
11
12         total=0
13
14         for worker in range(n):
15             total+=cost[worker][perm[worker]]
16
17         if total<minimum:
18             minimum=total
19             best=perm
20
21     return best,minimum
22
23
24 cost=[[3,10,7],
25        [8,5,12],
26        [4,6,9]]
```

Assignment: (2, 1, 0)
Total Cost: 16

=== Code Execution Successful ===